COMEN NC6

HL7 integráció

Programozói és üzemeltetési dokumentáció

Implementáció: F:\pcsofts\hl7

Verzió: 1.0 | Állapotfelvétel: 2026. október 30.

A dokumentum a meglévő Python fogadó megértéséhez, integrálásához és továbbfejlesztéséhez készült. Bemutatja a hálózati protokollt, az üzenetfeldolgozást, a modulok szerepét, az API-t, az adatmodellt és a tesztelési szempontokat.

A helyi forráskód leírása, nem gyártói megfelelőségi nyilatkozat. A minták szintetikusak; a készülék teljes körű validálása külön feladat.

Tartalom

1-6. Rendszerkép, hálózat, MLLP, HL7 és paraméterkódok

7-10. Feldolgozás, ACK, állapottár és HTTP API

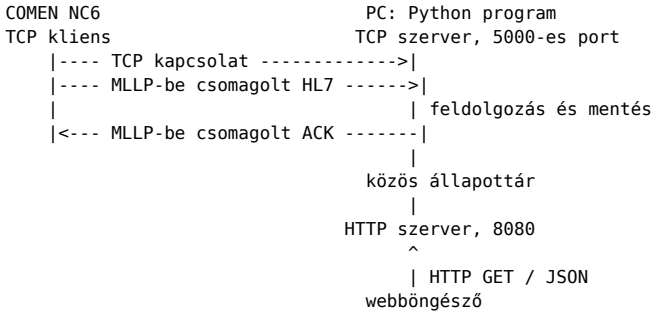
11-12. Tesztelés, hibakeresés és korlátok

13-16. Modulinterfészek, adatmodell, bővítés és regressziós ellenőrzések

Készült: 2026. október 30. Az ismertető az `F:\pcsofts\hl7` könyvtárban ellenőrzött forráskódot és konfigurációt írja le. Nem gyártói interfészspecifikáció. A példák mesterséges adatokat tartalmaznak.

1. Mit csinál a rendszer?

A COMEN monitor hálózaton elküldi a mérési üzeneteit a számítógépnek. A Python program fogadja és feldolgozza őket, fájlba menti az adatokat, visszaigazolást küld, és egy webfelületen megmutatja az eredményt.



A kapcsolatot a monitor kezdeményezi. A jelenlegi program nem kérdezi le aktívan a készüléket, nem indít vérnyomásmérést, és nem módosítja a monitor beállításait. Az adatküldés gyakoriságát a monitor működése és beállítása határozza meg.

2. Melyik protokoll mire szolgál?

Réteg	Feladat ebben a rendszerben
IP	Eljuttatja az adatot a megfelelő hálózati címre.
TCP	Rendezett bájtfolyamot biztosít a monitor és a PC között.
MLLP	Kijelöli a HL7 üzenetek elejét és végét a bájtfolyamban.
HL7 v2	Meghatározza az üzenetek szöveges felépítését és mezőit.
MDC kódok	Azonosítják az egyes mérési paramétereket és egyes mértékegységeket.
HTTP / JSON	A PC webfelülete és API-ja ezen keresztül szolgálja ki a böngészőt.

A monitor felőli 5000-es port nem weboldal: böngészővel a 8080-as portot kell megnyitni. Az 5000 és 8080 a program konfigurációja, nem a HL7 szabvány kötelező portszáma.

A projekt HL7 v2.6 profilra készült. A parser kiolvassa az üzenet verzióját, de nem végez teljes, verzióspecifikus HL7-ellenőrzést. Ez a megoldás a HL7 v2 szöveges üzeneteit kezeli; FHIR API nincs benne.

3. Hálózati beállítás és indítás

Jelenlegi konfiguráció:

Beállítás	Érték	Jelentés
`hl7_host`	`0.0.0.0`	A HL7 fogadó minden helyi IPv4 interfészen figyel.
`hl7_port`	`5000`	A monitor ide kapcsolódik.
`web_host`	`0.0.0.0`	A webfelület hálózatról is elérhető lehet.
`web_port`	`8080`	A böngészős felület portja.

Beállítás	Érték	Jelentés
`send_ack`	`true`	HL7 visszaigazolás küldése.
`save_raw_hl7`	`true`	Az eredeti üzenetek naplózása.
`history_size`	`1000`	Legfeljebb ennyi feldolgozott üzenet marad a memóriabeli előzményben.
`stale_seconds`	`15`	Frissességi küszöb másodpercben.
`encoding`	`utf-8`	Bejövő szöveg dekódolása.

A konfigurált eszköz-hozzárendelés: `192.168.0.118` → `KC574337852G`. Ez a monitor címe, nem a PC címe. A PC aktuális LAN IP-címét külön kell ellenőrizni, például az `ipconfig` paranccsal.

1. Indítsd el az `F:\pcsofts\hl7\run.bat` fájlt. Ez a saját könyvtárába lép, majd a `py -3 app.py`, illetve szükség esetén a `python app.py` parancsot futtatja.

2. A monitor HL7 célcíme a PC tényleges LAN IP-címe legyen, a célport pedig `5000`.

3. A PC tűzfalának engednie kell a monitor felől érkező TCP 5000 kapcsolatot.

4. Ugyanezen a PC-n nyisd meg: `http://127.0.0.1:8080`.

5. Másik számítógépről: `http://PC_LAN_IP:8080`; ehhez a TCP 8080 elérése is szükséges.

A `0.0.0.0` figyelési cím, a `127.0.0.1` pedig mindig a saját gépet jelenti. Egyiket sem kell a monitorban a PC hálózati célcímeként megadni. A monitor pontos menüútvonalát az adott firmware dokumentációja határozza meg; azt a Python kód nem tartalmazza.

Leállítás: Ctrl+C a program konzoljában. A konfiguráció módosítása után újra kell indítani a programot. Külön konfiguráció használható az `app.py --config config-lab.json` kapcsolóval.

4. Az MLLP keretezés

Egy teljes keret bájtszinten:

```
0B + HL7 üzenet + 1C 0D
```

A `0B` az üzenet kezdetét jelöli, az `1C 0D` a végét. Ezek valódi vezérlőbájtok, nem a „0B” felirat karakterei. A HL7 szegmensek között külön `0D` (CR) szerepel. HL7 MLLP leírás

A TCP nem őrzi meg az üzenethatárokat: egy üzenet több fogadási darabra szakadhat, és több üzenet is érkezhet egyetlen darabban. A `hl7\mllp.py` ezért puffereli a beérkező bájtokat. Csak teljes keretet ad tovább; több keretet különválaszt, hibás kezdés után újraszinkronizál, és körülbelül 1 MiB-os keretméret-korlátot alkalmaz.

5. Egy HL7 üzenet olvasása

A HL7 v2 üzenet szegmensekből áll. A szokásos mezőelválasztó a `|`, a mezőn belüli komponenseké a `^`. Az `MSH` fejléc, az `OBX` mérési adatot hordoz. Az `ORU^R01` megfigyelési eredmények kérés nélküli továbbítására szolgál. HL7 megfigyelési üzenetek

Az alábbi rövidített példa a helyi parser szemléltetésére készült, nem teljes gyártói mintaprofil. A látható sortörések a hálózaton CR karakterek:

```
MSH|^~\&|NC_Monitor|DEMO_NC6|RECEIVER|LOCAL|20261030120000||ORU^R01^ORU_R01|DEMO001|P|2.6
OBX|1|NM|150021^MDC_PRESS_BLD_NONINV_SYS^MDC||126|266016^MDC_DIM_MMHG^MDC|||||F
OBX|2|NM|150022^MDC_PRESS_BLD_NONINV_DIA^MDC||78|266016^MDC_DIM_MMHG^MDC|||||F
```

Ebben a helyi program `DEMO001` üzenetazonosítót, `126`-os szisztolés és `78`-as diasztolés értéket olvas ki. A `266016` egységkódot `mmHg` felirattá alakítja. Számérték-átváltást nem végez.

Mező	Mit olvas ki belőle a program?
MSH-3, MSH-4	Küldő alkalmazás és intézmény mező; az észlelt NC_Monitor profilban az MSH-4 eszközazonosítóként is használható.
MSH-9	Üzenettípus; a mérésfeldolgozó `ORU^R01` típust fogad el.
MSH-10	Üzenetazonosító, amely az ACK összerendeléséhez kell.
MSH-12	HL7 verzió.
OBX-2	Adattípus; az `NM` értéket számmá alakítja.
OBX-3	A paraméter kódja, neve és kódrendszere.
OBX-4	Megfigyelés alazonosítója.
OBX-5	Érték.
OBX-6	Mértékegység.
OBX-11, OBX-14	Státusz és mérési idő; tárolódnak.
OBX-18	Eszközazonosítás egyik lehetséges forrása.

A feldolgozó az MSH és OBX tartalmára épül. A PID/OBR adatokat nem alakítja beteg- vagy vizsgálati nyilvántartássá. Az eredeti üzenet ugyanakkor a nyers naplóban megmaradhat.

6. A jelenlegi paraméterkódok

Az alábbiak a helyi `hl7\codes.py` hozzárendelései; nem teljes gyártói vagy MDC kódtár.

Kód	Belső név	Megjelenített paraméter
150456	`spo2`	SpO_2
149522	`pulse_rate`	Pulzus
150021 / 150301	`nibp_sys`	Szisztolés vérnyomás
150022 / 150302	`nibp_dia`	Diasztolés vérnyomás
150023 / 150303	`nibp_mean`	Átlagos artériás nyomás
149546	`nibp_pulse`	Vérnyomásméréshez tartozó pulzus
147842	`ecg_heart_rate`	EKG szívfrekvencia

A kód megjegyzése szerint a 150021/150022/150023 az NC6 nyers üzeneteiből igazolt hozzárendelés. A 150301/150302/150303 korábbi hozzárendelésként maradt benne; a szimulátor ezeket használja. A kódban szereplés önmagában nem jelenti az összes paraméter valódi készülékes validálását.

Egységfeliratok: 262688 → `%`, 264864 → `bpm`, 266016 → `mmHg`, ha az egység kódrendszere MDC. Más kódrendszer egységkódját a program nem értelmezi automatikusan MDC-ként.

Az ismeretlen paraméterek az `observations` listában megmaradnak. Hőmérséklethez és légzésszámhoz van kijelzőkártya, de a jelenlegi `extra_codes` üres: a valódi eszközkódokhoz külön hozzárendelés szükséges. A szimulátor `SIM_TEMP` és `SIM_RESP` azonosítói kizárólag tesztkódok.

7. A szerver belső folyamata

1. Az app.py beolvassa a konfigurációt, létrehozza az állapottárat és a két szervert.

2. A HL7 szerver külön szálon fut; minden TCP kapcsolatot külön kezelőszál szolgál ki.

3. A hl7\server.py bájtokat fogad, és átadja őket az MLLP dekódernek.

4. A teljes üzenetet a beállított karakterkódolással szöveggé alakítja. Hibás karakterek helyettesítő jellel kerülnek be.

5. Elmenti a nyers üzenetet, majd a hl7\parser.py feldolgozza a fejlécet és az OBX sorokat.

6. A state.py hozzáfűzi az eredményt a mérési fájlhoz, frissíti az előzményt és a legutóbbi értékeket.

7. Siker esetén AA, feldolgozási vagy mentési hiba esetén AE ACK készül, ha a fejléc alapján ez lehetséges.

8. A böngésző HTTP lekérdezéseken keresztül kapja meg az aktuális állapotot.

A TCP fogadás 1 másodperces időkorlátja csak a leállítás rendszeres ellenőrzését segíti: egy másodperc csend nem bontja a kapcsolatot. TCP keepalive is engedélyezett, de a hálózati hiba felismerésének ideje az operációs rendszer beállításaitól is függ.

Az eszközazonosítás sorrendje: konfigurált IP-hozzárendelés → OBX-18 → NC_Monitor esetén MSH-4 → MSH-3 → IP-cím.

8. Visszaigazolás: ACK

A program az ACK-ot ugyanazon TCP kapcsolaton, MLLP keretben küldi vissza. Az MSA második adatmezőjében az eredeti üzenetazonosító szerepel; ebből derül ki, melyik üzenetre vonatkozik a válasz.
HL7 ORU és ACK kapcsolat

```
MSA|AA|DEMO001
```

A helyi szerverben az AA azt jelzi, hogy a feldolgozás és a fájlírás sikeresen lefutott. Nem jelent klinikai ellenőrzést, és nem garantál áramszünetbiztos adattárolást: külön lemezes szinkronizálás nincs.

Az AE hibát jelez. Ilyet kap a nem támogatott üzenettípus, például az R40 is. Ha az MSH fejléc hiányzik vagy az üzenetazonosító nem olvasható, a program nem készít korrelált ACK-ot, hanem naplózza a hibát. Az ACK kikapcsolható a konfigurációban.

A program nem valósítja meg a kibővített commit ACK módot, és nem szűri ki az ismételt üzenetazonosítókat. Ha a monitor újraküld egy mérést, az ismét bekerülhet az előzménybe. A monitor újraküldési viselkedését külön kell megismerni.

9. Webfelület, frissesség és tárolás

A felület nagyjából másodpercenként lekéri az adatokat; pontosabban minden frissítési kör befejezése után egy másodpercet vár. Nem WebSocket-alapú adatfolyam.

A TCP kapcsolat fennállása és az adatok frissessége külön fogalom. A frissesség határa jelenleg 15 másodperc. Minden kijelzett paraméter saját fogadási időt kap: egy új hullámformaüzenet nem teszi frissebbé a korábbi vérnyomásértéket. A PC fogadási ideje alapján számol, nem az OBX-14 mérési időből.

A Live nézet megőrzi az egyes paraméterek utolsó értékét. A History viszont üzenetenkénti pillanatkép: az adott üzenetből hiányzó értékeket nem tölti ki korábbi mérésekkel. Ugyanazon paraméter többszöri előfordulásakor a kártyára az üzeneten belüli utolsó érték kerül.

Fájl	Tartalom
F:\pcsofts\hl7\data\hl7_raw.log	Eredeti üzenet, fogadási idő és küldő IP, JSON-soronként.

Fájl	Tartalom
`F:\pcsofts\hl7\data\measurements.jsonl`	Feldolgozott üzenetek és megfigyelések, JSON-soronként.
`F:\pcsofts\hl7\logs\app.log`	Kapcsolatok, elfogadások, hibák; 5 MB-os rotáció és három mentett napló.

Induláskor a program az előzményfájlból az utolsó legfeljebb 1000 bejegyzést tartja meg. A Live nézet új beérkező üzenetre vár. A Raw nézet az aktuális futás utolsó 200 üzenetét mutatja. A két adatfájl mérete folyamatosan nő; ezekre nincs automatikus rotáció.

10. HTTP API más programok számára

Végpont	Eredmény
`/api/health`	Alkalmazásállapot és figyelési állapotjelző; nem teljes végponttól végpontig tartó kommunikációteszt.
`/api/devices`	Ismert eszközök, állapotok és értékek.
`/api/latest`	Legutóbbi eszközállapot.
`/api/history`	Memóriában lévő mérési előzmény.
`/api/history.csv`	CSV-export: idő, eszköz, SpO_2, pulzus, SYS/DIA/MAP és egységek.
`/api/raw`	Legfeljebb 200 nyers üzenet az aktuális futásból.
`/api/settings`	Aktuális konfiguráció, csak olvasásra.

Példák:

```
http://127.0.0.1:8080/api/latest?device_id=KC574337852G
http://127.0.0.1:8080/api/history?device_id=KC574337852G&limit=100
```

A `device_id` szűrés a latest/history/CSV végpontoknál használható. A `limit` az előzmény és CSV méretét korlátozza 1 és 1000 között. A CSV és az előzménytáblázat nem tartalmaz minden lehetséges paramétert; a JSON részletesebb.

11. Kipróbálás és hibakeresés

Elindított szerver mellett, külön konzolból:

```
Set-Location F:\pcsofts\hl7
py -3 simulator.py --device SIM_TEST --count 10 --fragment --batch 2
```

Ez mesterséges adatot küld a helyi fogadóhoz, ellenőrzi az ACK-ot és az üzenetazonosítót, darabolt TCP-küldést is kipróbál. A `SIM_TEST` név megkülönbözteti a szimulátort a valódi monitortól. A tesztadatok a szerver adatfájljaiba is bekerülnek.

Az automatizált tesztek indítása:

```
py -3 -m unittest discover -s tests -v
```

E dokumentáció elkészítésekor a teszteket nem futtattuk újra, és nem kezdeményeztünk új készülékes mérést.

Jelenség	Ellenőrzés
Nem indul a program	Python elérhető-e; a konzol hibája; foglalt-e az 5000 vagy 8080 port?

Jelenség	Ellenőrzés
A weboldal működik, adat nincs	Monitor cél-IP/port, adatküldés engedélyezése, tűzfal, `app.log`.
Kapcsolódik, de régi adat látható	Érkezik-e megfelelő OBX paraméter; nem csak más típusú adatot küld-e?
Raw üzenet van, kártya nincs	OBX-3 kód/kódrendszer, OBX-2 adattípus, `extra_codes` hozzárendelés.
R40 hibák szerepelnek	Az R40 feldolgozása jelenleg nincs megvalósítva.
Másik PC-ről nincs webelérés	Helyes PC-IP, TCP 8080 tűzfalszabály, web_host beállítás.

12. Jelenlegi korlátok és a következő ellenőrzések

A fogadó az ORU^R01 egy részhalmazát dolgozza fel, nem teljes HL7 validátor. A nem numerikus OBX tartalmakat alapvetően nyers értékként őrzi meg; teljes hullámforma-dekódolás és grafikon nincs. R40 esemény- és riasztásfeldolgozás sincs.

Nincs belépés, jogosultságkezelés vagy TLS. A jelenlegi `0.0.0.0` webbeállítás mellett a felület a tűzfal által engedett hálózatról elérhető. Helyi használathoz `web_host: "127.0.0.1"` állítható be. A nyers üzenetek betegazonosító adatokat is tartalmazhatnak; a fájlokhoz és hálózati portokhoz való hozzáférést ennek megfelelően kell korlátozni. A program megjelenítő és adatgyűjtő; nem helyettesíti a monitor riasztásait.

A régebbi README-nél tovább jutott a forráskód: a valódi NC6 vérnyomáskódjai, az egységkódok és egy hozzájuk tartozó teszt már szerepelnek benne. A teljes készülékes validáláshoz paraméterenként össze kell vetni a monitor kijelzőjét, a nyers OBX üzenetet és a webes értéket. Külön ellenőrizendő az újracsatlakozás, az ACK utáni viselkedés, a hőmérséklet/légzésszám kódja és az R40 kezelésének igénye.

Az ismertető elsődleges forrása a helyi `app.py`, `state.py`, `config.json`, `hl7\*.py`, `web\server.py`, `web\static\app.js`, `simulator.py` és `tests\test_nc6_blood_pressure.py`. A hivatkozott HL7 oldalak az általános protokollt magyarázzák; nem igazolják a COMEN firmware konkrét beállításait.

13. Modulok és belső interfészek

Modul / hívás	Szerződés és felelősség
app.py: load_config(path), main()	Konfiguráció beolvasása és részleges ellenőrzése; naplózás; két listener indítása és leállítása.
hl7/mllp.py: Decoder.feed(chunk)	Bájtdarab be, nulla vagy több teljes payload ki. Kapcsolatonként külön Decoder kell. A méretkorlát túllépése ValueError.
hl7/mllp.py: frame(message)	Szöveg vagy bájtok MLLP keretezése. A szöveges kimenet UTF-8.
hl7/parser.py: parse(text, ip, config)	Egy ORU^R01 feldolgozása; strukturált dict visszaadása; hibás tartalomnál ValueError.
hl7/parser.py: ack(text, code)	MSH+MSA válaszszöveg készítése; az MLLP keretezés külön lépés.
hl7/server.py: Handler.handle()	Kapcsolatkezelés, dekódolás, raw mentés, parse, accept és ACK.
state.py: State.accept(row, token)	Fájlba írás és a history/latest frissítése a közös zárolás alatt.
state.py: State.devices()	Másolatot készít az eszközállapotokról; kiszámítja a frissességet.
web/server.py: make_server(...)	Többszálú HTTP szerver; fix statikus útvonalak és GET API.
web/static/app.js	Lekérdezési ciklus, eszközválasztás, kártyák, raw és history nézet.

A State egy RLock zárral védi a megosztott állapotot. A fájlírás is a zárolt szakaszban történik: egyszerű és követhető megoldás, de nagy adatforgalomnál a lemezműveletek feltarthatják a többi kapcsolatot. A lekérdezések mély másolatot adnak vissza, így a HTTP kiszolgálás nem módosítja közvetlenül a belső adatot.

A feldolgozás sorrendje lényeges: raw mentés → parse → mérési fájl írás → memóriafrissítés → ACK. Nincs több fájlra kiterjedő tranzakció. Hiba esetén lehet olyan raw sor, amelyhez nem tartozik elfogadott mérés. ACK-küldési hiba esetén a mérés már el lehet mentve.

14. Adatmodell és integrációs szerződés

A parse eredménye üzenetenként egy rekord. A vitals a felismert paramétereket név szerint tartalmazza, az observations minden feldolgozott OBX-et listáz, beleértve az ismeretlen kódokat is.


```
{
  "device_id": "DEMO_NC6",
  "device_ip": "192.0.2.10",
  "hl7_version": "2.6",
  "message_id": "DEMO001",
  "received_at": "2026-10-30T12:00:00+01:00",
  "vitals": {
    "nibp_sys": {
      "code": "150021",
      "name": "MDC_PRESS_BLD_NONINV_SYS",
      "value": 126,
      "unit": "mmHg",
      "status": "F",
      "observed_at": "",
      "sub_id": "",
      "data_type": "NM",
      "raw_value": "126"
    }
  },
  "observations": ["itt a teljes OBX objektumok szerepelnek"]
}
```

A példa szerkezeti szemléltetés: az observations tényleges elemei a fenti mérési objektumhoz hasonló objektumok, nem szövegek. A latest/devices API a rekordhoz connected, fresh és age_seconds mezőt ad. A vitals elemei ezen felül saját received_at, fresh és age_seconds mezőt kapnak. Ezek az API-ban számított adatok nem azonosak a fájlban tárolt üzenetpillanatképpel.

NM típusnál az egész érték int, a többi véges numerikus érték float. Üres vagy HL7 null értéknél a value null. Nem numerikus típusnál jellemzően szöveg marad. NaN és végtelen érték hibát okoz. A kliens ezért nem feltételezheti, hogy minden value szám.

Az observations egységmezője eltérhet a vitals egységétől: hiányzó küldött egységnél a vitals a kódtábla alapértelmezését kapja. Az OBX-11 státusz megőrződik, de a jelenlegi program nem szűri a megjelenítést a státusz alapján.

A kódrendszer a hozzárendelés eldöntéséhez használatos, de önálló namespace mezőként nem marad meg a strukturált megfigyelésben. A teljes eredeti azonosító a raw üzenetből állítható vissza. A több OBR/PID csoport hierarchiáját a parser nem őrzi meg; beteghez kötött integrációhoz ezt külön kell megvalósítani.

Az API nem kínál lapozást vagy időintervallum szerinti történeti keresést. A limit az aktuális memóriabeli előzmény utolsó elemeit választja ki. Nincs POST/PUT konfigurációmódosítás, OpenAPI séma vagy verziózott API-útvonal.

15. Bővítési útmutató

Új mérési paraméterhez először valódi vagy gyártó által igazolt OBX mintát kell gyűjteni. Ellenőrizni kell a kódot, kódrendszert, adattípust, egységet, státuszt és a mérés időpontját. Ezután a hozzárendelés a codes.py állományba vagy az extra_codes konfigurációba kerülhet.

```
"extra_codes": {
  "SIM_TEMP": {"key": "temperature", "unit": "Cel"}
}
```

Ez kizárólag szimulátoros példa. A SIM_TEMP helyére csak ellenőrzött valódi eszközkód kerüljön. Az extra_codes a jelenlegi parser kódrendszer-szűrését nem kerüli meg: LOCAL vagy más nem támogatott namespace esetén a hozzárendelés nem aktiválódik.

Ha új belső kulcsot vezetsz be, a web/static/app.js fields listáját is bővíteni kell. A CSV saját, rögzített keys listával működik, a history táblázat pedig csak az első öt kijelzési mezőt használja; ezeket külön kell módosítani. Az extra_codes önmagában nem hoz létre új kártyát vagy CSV-oszlopot.

Új üzenettípushoz külön feldolgozási útvonal és megfelelő ACK-viselkedés szükséges. R40 támogatásakor nem elegendő az ORU^R01 feltétel lazítása: az események tartalmát, tárolását és felhasználói megjelenítését is definiálni kell.

Nagyobb terheléshez megfontolandó egy írási sor és adatbázis bevezetése. Ilyenkor előre rögzíteni kell, hogy az AA ACK a sorba állítást vagy a tartós mentést igazolja. A jelenlegi feldolgozási garanciát nem szabad észrevétlenül megváltoztatni.

Duplikációszűréshez eszköz- és üzenetazonosítóból képzett kulcs, megfelelő megőrzési idő és újraindítást túlélő tároló szükséges. A monitor tényleges azonosító-újrafelhasználását előtte ellenőrizni kell.

16. Regressziós és átvételi ellenőrzések

Terület	Elvárt ellenőrzés
MLLP	Minden vágási pont, bájtonként érkezés, több keret egy darabban, hibás keret utáni helyreállás, méretkorlát.
Parser	Ismert és ismeretlen kód, más namespace, üres érték, hibás NM, ismétlődő paraméter, eltérő egység.
ACK	AA/AE, MSA-2 egyezés, feladó/címzett felcserélése, hibás MSH, ACK nélküli konfiguráció.
Tárolás	Mentési hiba, újraindítás, sérült history sor, korlátos memória, ismételt üzenet.
Állapot	Több eszköz, újracsatlakozás, TCP csend, paraméterenkénti frissesség.
API és UI	Eszközszűrés, limit hibák, null értékek, HTTP hiba, CSV oszlopok és egységek.
Valódi NC6	Monitor kijelzője ↔ OBX ↔ webes érték; mérések időzítése; újraküldés és kapcsolatvesztés.

A táblázat fejlesztői ellenőrzőlista, nem állítás arról, hogy minden esetet már lefednek a meglévő tesztek. A dokumentáció készítésekor új tesztfutás és készülékes validálás nem történt.